SAP Fiori Elements with CAP OData V4

CDS Code Examples

Session 4 · UX405 · C_FIORD Exam Preparation · Companion Code Guide

Builds on the Demonstration Evidence Log | Full CAP Node.js Project Walkthrough

Fiori Elements & FPM: 21–30% | Annotations: 11–20% | OData & BTP: 11–20%

cap.cloud.sap/docs · learning.sap.com/courses/developing-an-sap-fiori-elements-app-based-on-a-cap-odata-v4-service

---

Purpose of This Guide

This companion document turns the annotation fragments already captured in your Demonstration Evidence Log into one complete, working CAP CDS project. It follows the same session structure and visual style as the Session 4 Study Guide, so you can study them side by side.

Each code block is written for SAP Cloud Application Programming Model (CAP) with Node.js, targeting OData V4 — the only combination UX405 and the C_FIORD exam test. Every annotation is cross-checked against SAP's official CAP documentation (cap.cloud.sap/docs) and the SAPUI5 Fiori Elements annotation reference.

How to Use This Guide

- Read alongside the Study Guide — each part below is labelled with the matching UX405 unit numbers (U1–U14).
- Type the code into your own SAP Business Application Studio (BAS) CAP project to reinforce muscle memory before the exam.
- Where your evidence log used a different (ABAP CDS / RAP) syntax for the same concept, this is called out explicitly — see the trap box below.
- Paste your own working code and screenshots into the 'Your Code' sections of the evidence log once you've reproduced these examples.

⚠ Exam Trap ★

Your Demonstration Evidence Log's Session 4 snippets (@UI.selectionField, @UI.lineItem, @UI.facet with parentId/targetQualifier, CASE WHEN criticality on sf.price) are written in ABAP CDS / RAP annotation syntax — lowercase, singular terms, used for on-premise ABAP Gateway or RAP-based Fiori apps.
UX405 and C_FIORD test CAP CDS syntax for OData V4 — uppercase, plural terms (@UI.SelectionFields, @UI.LineItem, @UI.Facets), written in .cds files on SAP BTP. The concepts are identical; the syntax is not. This guide rebuilds every old example in the correct CAP syntax below.

ABAP CDS (RAP) vs CAP CDS — Syntax Comparison

Same UX405 exam concept, two different implementation worlds. Know which one a question is asking about — CAP + 'BTP' + 'OData V4' always points to the right-hand column.

Concept	Your Evidence Log (ABAP CDS / RAP)	CAP CDS for UX405 (this guide)
Filter bar field	@UI.selectionField: [{ position: 10 }]	@UI.SelectionFields: [carrierID, ...]
Table column	@UI.lineItem: [{ position: 80, label:'Price' }]	@UI.LineItem: [{ Value: price, Label:'Price' }]
Criticality source	CASE WHEN ... END AS PriceCriticality	Calculated element: priceCriticality = case when...
Object Page section	@UI.facet: [{ id, parentId, targetQualifier }]	@UI.Facets: [{ $Type:'UI.ReferenceFacet', Target }]
Where it's defined	define view entity ... (ABAP repository)	.cds files under db/ and srv/ (BTP project)
OData version	Usually V2 via SAP Gateway	Always V4 — CAP does not support V2

The Business Scenario for This Guide

To keep continuity with the flight-themed data (carrid / connid / price) already in your evidence log, the examples below model a Flights entity in CAP — the same business idea, rebuilt as a proper CAP OData V4 service. Field names are re-cased to CAP convention (camelCase) as SAP's own CAP samples do.

New CDS Project Structure

File	Purpose	UX405 Units Covered
db/schema.cds	Core data model — entities, fields, calculated elements	U1, U6–U7, U11
srv/flight-service.cds	OData V4 service definition, draft, bound/unbound actions	U1, U4, U6, U8
srv/flight-service-ui.cds	Fiori Elements UI annotations (@UI.*)	U5–U10, U13–U14
srv/flight-service.js	Node.js service handlers — validation, side effects	U11
webapp/ext/ (FPM fragment)	Custom section XML fragment + manifest.json entry	U12

U1–U3: Data Model + Service Definition

Foundation for every other annotation in this guide

db/schema.cds — Core Data Model (U1)

The calculated element priceCriticality replaces your evidence log's CASE WHEN column directly — same logic, CAP calculated-element syntax.

```
namespace sap.fe.demo.flight;

using { cuid, managed } from '@sap/cds/common';

entity Flights : cuid, managed {
  carrierID     : String(3)    @title: 'Carrier';
  connectionID  : String(4)    @title: 'Connection';
  flightDate    : Date         @title: 'Flight Date';
  originAirport : String(3)    @title: 'From';
  destAirport   : String(3)    @title: 'To';
  price         : Decimal(9,2) @title: 'Price';
  currency_code : String(3)    @title: 'Currency' default 'USD';
  plannedSeats  : Integer      @title: 'Planned Seats';
  bookedSeats   : Integer      @title: 'Booked Seats';
  status        : String(1)    @title: 'Status' default 'A';
  // A = Open, X = Closed, C = Cancelled

  // Calculated element — converts your old ABAP CDS
  // CASE WHEN price bracket logic into CAP CXL syntax
  priceCriticality : Integer = case
    when price <= 500  then 3   // green
    when price <= 1000 then 2   // amber
    else 1                      // red
  end;
}
```

srv/flight-service.cds — OData V4 Service (U1, U4)

CAP auto-generates the OData V4 metadata and mock server from this projection — no manual EDMX, no ABAP Gateway registration required.

```
using { sap.fe.demo.flight as db } from '../db/schema';

@path: '/odata/v4/flight'
service FlightService {

  @odata.draft.enabled
  entity Flights as projection on db.Flights;

}
```

SAP Reference

CAP project layout and cds init: cap.cloud.sap/docs/get-started/
Calculated elements (CXL): cap.cloud.sap/docs/cds/cxl#calculated-elements

U4–U5: Filter Bar Annotations

Converts your evidence log's @UI.selectionField list into CAP's array-based @UI.SelectionFields

srv/flight-service-ui.cds — Filter Bar (U5)

```
using { FlightService } from './flight-service';

annotate FlightService.Flights with @(
  UI.SelectionFields: [
    carrierID,
    connectionID,
    status
  ]
);

// Free-text search box in the filter bar
annotate FlightService.Flights with @(
  Search.searchable: true
);
```

Default Filter Values (U5)

@UI.SelectionPresentationVariant sets a pre-applied filter — the CAP equivalent of the old @UI.SelectionVariant default value pattern.

```
annotate FlightService.Flights with @(
  UI.SelectionPresentationVariant #openOnly: {
    SelectOptions: [{
      PropertyName: status,
      Ranges: [{ Sign: #I, Option: #EQ, Low: 'A' }]
    }],
    Text: 'Open Flights',
    Visualizations: ['@UI.LineItem']
  }
);
```

> **⚠ Exam Trap ★**
> @UI.SelectionFields always takes an array of property names directly — never wrap them in { position: n } objects the way ABAP CDS @UI.selectionField does. That objectstyle only exists in the ABAP RAP world.

U6–U7: Actions + Table Column Criticality

Bound vs unbound actions, plus the CAP rewrite of your CASE WHEN criticality column

srv/flight-service.cds — Bound & Unbound Actions (U6)

cancelFlight is bound — declared inside actions{} on the entity, so it only runs against one selected Flights record. openBookingWindow is unbound — declared at service level, so it runs independently of any selection.

```
service FlightService {

  @odata.draft.enabled
  entity Flights as projection on db.Flights actions {
    // BOUND action — needs a selected Flights record
    @(Common.IsActionCritical: true)
    action cancelFlight() returns Flights;
  };

  // UNBOUND action — no row selection required
  action openBookingWindow(forDate: Date) returns String;

}
```

Toolbar Buttons for Both Actions (U6)

```
annotate FlightService.Flights with @(
  UI.LineItem: [
    { Value: carrierID },
    { Value: connectionID },
    { Value: price,
      Criticality: priceCriticality },
    { $Type: 'UI.DataFieldForAction',
      Action: 'FlightService.cancelFlight',
      Label: 'Cancel Flight' },        // BOUND → inline
    { $Type: 'UI.DataFieldForAction',
      Action: 'FlightService.openBookingWindow',
      Label: 'Open Booking Window' }   // UNBOUND → toolbar
  ]
);
```

Key Exam Point

Criticality: priceCriticality references the calculated element built on db/schema.cds — CAP resolves the numeric value (1/2/3) directly at the LineItem column, exactly matching your old CASE WHEN → PriceCriticality → @UI.lineItem.criticality chain, just in CAP syntax.

U8: Multiple Views + Side Effects

Tab views built on @UI.SelectionPresentationVariant, and an annotation-driven auto-refresh

Multiple Table Views — Tab Bar (U8)

```
annotate FlightService.Flights with @(
  UI.SelectionPresentationVariant #openFlights: {
    SelectOptions: [{ PropertyName: status,
      Ranges: [{ Sign: #I, Option: #EQ, Low: 'A' }] }],
    Text: 'Open',
    Visualizations: ['@UI.LineItem']
  },
  UI.SelectionPresentationVariant #closedFlights: {
    SelectOptions: [{ PropertyName: status,
      Ranges: [{ Sign: #I, Option: #EQ, Low: 'X' }] }],
    Text: 'Closed',
    Visualizations: ['@UI.LineItem']
  }
);
```

@Common.SideEffects — Auto-Refresh Booked Seats (U8)

Changing bookedSeats automatically re-fetches price and priceCriticality — declared purely as an annotation, never as a JavaScript event handler.

```
annotate FlightService.Flights with @(
  Common.SideEffects #SeatsChanged: {
    SourceProperties: [ bookedSeats ],
    TargetProperties: [ price, priceCriticality ]
  }
);
```

SAP Reference

Side Effects guide: cap.cloud.sap/docs/advanced/fiori#side-effects
UX405 unit: 'Configuring Side Effects' — learning.sap.com/courses/developing-an-sap-fiori-elements-app-based-on-a-cap-odata-v4-service

U9–U10: Object Page Header + Body

Rebuilds your evidence log's facet with parentId/targetQualifier using @UI.Facets + @UI.FieldGroup

@UI.HeaderInfo — Object Page Title (U9)

```
annotate FlightService.Flights with @(
  UI.HeaderInfo: {
    TypeName: 'Flight',
    TypeNamePlural: 'Flights',
    Title: { Value: connectionID },
    Description: { Value: carrierID }
  }
);
```

@UI.Facets + @UI.FieldGroup — Body Sections (U10)

The old ABAP CDS facet used id / parentId / targetQualifier to nest a FieldGroup inside a section. CAP expresses the same nesting with CollectionFacet → ReferenceFacet → Target.

```
annotate FlightService.Flights with @(
  UI.Facets: [{
    $Type: 'UI.CollectionFacet',
    ID: 'FlightDetails',
    Label: 'Flight Details',
    Facets: [{
      $Type: 'UI.ReferenceFacet',
      ID: 'GeneralData',
      Label: 'General Data',
      Target: '@UI.FieldGroup#GeneralData'
    }]
  }],
  UI.FieldGroup #GeneralData: {
    Data: [
      { Value: originAirport },
      { Value: destAirport },
      { Value: flightDate },
      { Value: plannedSeats },
      { Value: bookedSeats }
    ]
  }
);
```

> **⚠ Exam Trap ★**
>
> There is no parentId or targetQualifier in CAP CDS. Nesting is expressed structurally: a CollectionFacet contains a Facets array, and each ReferenceFacet points to its content via Target — always a string reference to the annotation term, e.g. '@UI.FieldGroup#GeneralData'.

U11: Dynamic Field Control + Input Validation

Field-level editability rules, plus a CAP Node.js validation handler

Dynamic Editability with a Calculated Element (U11)

A calculated field control drives editability from the status field — mandatory while a flight is open, read-only once closed.

```
// Added to db/schema.cds
entity Flights : cuid, managed {
  // ...existing fields...
  statusFieldControl : Integer = case
    when status = 'A' then 7   // Mandatory while Open
    when status = 'X' then 1   // ReadOnly once Closed
    else 3                     // Optional otherwise
  end;
}

annotate FlightService.Flights with {
  plannedSeats @Common.FieldControl: statusFieldControl;
};
```

srv/flight-service.js — Before-Save Validation (U11)

Business validation always lives in the CAP service handler — never in the UI layer. req.error() renders the message inline on the field automatically.

```
const cds = require('@sap/cds');

module.exports = cds.service.impl(async function () {
  const { Flights } = this.entities;

  this.before('CREATE', Flights, (req) => {
    if (req.data.bookedSeats > req.data.plannedSeats) {
      req.error(400, 'Booked seats cannot exceed planned seats');
    }
  });

  this.on('cancelFlight', Flights, async (req) => {
    const flight = req.params[0];
    await UPDATE(Flights, flight.ID).with({ status: 'C' });
    return req.data;
  });
});
```

SAP Reference

Providing Services (before/on/after handlers): cap.cloud.sap/docs/guides/providing-services
Field Control values (0/1/3/7) — SAPUI5 UI annotation vocabulary:
sapui5.hana.ondemand.com/sdk/#/topic/7619517a924f4e859801e3a5d6a5d9f5

U12: Flexible Programming Model (FPM)

Custom section added alongside the generated Object Page — annotations still come first

XML Fragment — Custom Section Content (U12)

Building Blocks bring FE-managed controls (macros:Table) into hand-written XML, keeping a consistent look with the rest of the annotation-driven app.

```
<!-- webapp/ext/fragment/BookingHistory.fragment.xml -->
<core:FragmentDefinition
  xmlns:core="sap.ui.core"
  xmlns:macros="sap.fe.macros">
  <macros:Table
    id="bookingHistoryTable"
    metaPath="@com.sap.vocabularies.UI.v1.LineItem"
    contextPath="/Flights" />
</core:FragmentDefinition>
```

manifest.json — Registering the Custom Section (U12)

```
"content": {
  "body": {
    "sections": {
      "BookingHistorySection": {
        "template": "sap.fe.demo.flight.ext.fragment.BookingHistory",
        "title": "Booking History",
        "position": { "placement": "After",
                      "anchor": "FlightDetails" }
      }
    }
  }
}
```

Key Exam Point

Key rule for the exam: reach for annotations first. FPM custom sections are only justified when the requirement genuinely cannot be expressed with @UI.Facets, @UI.FieldGroup, or the other annotation terms covered on the previous pages.

U13–U14: Navigation + Analytical List Page

Intent-based navigation, and the HANA-only ALP chart annotation

Intent-Based Navigation (U13)

```
annotate FlightService.Flights with @(
  UI.LineItem: [
    { Value: carrierID },
    { $Type: 'UI.DataFieldWithIntentBasedNavigation',
      Value: connectionID,
      SemanticObject: 'Flight',
      Action: 'displayFactSheet' }
  ]
);
```

@UI.Chart — Analytical List Page (U14)

Included for completeness against the exam objective — remember ALP requires SAP HANA as the database, unlike every other example in this guide, which runs on Any DB.

```
annotate FlightService.Flights with @(
  UI.Chart: {
    ChartType: #Bar,
    Dimensions: [ carrierID ],
    Measures: [ price ],
    MeasureAttributes: [{
      Measure: price,
      Role: #Axis1
    }]
  }
);
```

> **⚠ Exam Trap ★**
> ALP = SAP HANA only. If this Flights service were deployed on a non-HANA database (e.g. SQLite for local dev, PostgreSQL), the List Report + Object Page template still works — the Analytical List Page template will not.

🎯 Know These Cold — CDS Code Reference

File	Annotation / Construct	What It Does
db/schema.cds	field = case when...end	Calculated element — e.g. priceCriticality
srv/*.cds	@odata.draft.enabled	Enables draft handling on the projection
srv/*.cds	actions { action x(); }	Bound action — needs a selected record
srv/*.cds	action x(); (service level)	Unbound action — no selection needed
*-ui.cds	@UI.SelectionFields	Filter bar fields
*-ui.cds	@UI.LineItem	List Report / sub-table columns
*-ui.cds	@UI.Facets / @UI.FieldGroup	Object Page body structure
*-ui.cds	@UI.HeaderInfo	Object Page title / description
*-ui.cds	@Common.SideEffects	Annotation-driven auto-refresh
*-ui.cds	@Common.FieldControl	Dynamic editability (0/1/3/7)
*.js	req.before('CREATE', ...)	Server-side validation, req.error()

Common Syntax Pitfalls — CAP CDS

- Property arrays are square brackets [ ] — not the { position: n } object style from ABAP CDS.
- Calculated elements use = directly on the field, not virtual with a separate CASE clause elsewhere.
- Bound actions live inside actions { } on the entity projection; unbound actions sit at service level.
- Criticality on @UI.LineItem is a direct property reference (Criticality: fieldName) — not a nested DataPoint unless you also need a progress bar or rating control.
- Always run cds watch locally to compile-check annotations before assuming exam syntax is correct — the CAP compiler will fail loudly on invalid CDL.

SAP Resources for This Code Guide

CAP Documentation (capire): https://cap.cloud.sap/docs

Serving SAP Fiori UIs from CAP: https://cap.cloud.sap/docs/guides/uis/fiori

Providing Services (handlers, req.before/on/after): https://cap.cloud.sap/docs/guides/providing-services

CDS Expression Language (calculated elements): https://cap.cloud.sap/docs/cds/cxl

Common Annotations Reference: https://cap.cloud.sap/docs/cds/annotations

Fiori Elements V4 Annotation Reference (SAPUI5 SDK): https://sapui5.hana.ondemand.com/sdk/#/topic/7619517a924f4e859801e3a5d6a5d9f5

UX405 Course (SAP Learning): https://learning.sap.com/courses/developing-an-sap-fiori-elements-app-based-on-a-cap-odata-v4-service

SAP Fiori Tools Documentation: https://help.sap.com/docs/sap-fiori-tools

SAP BTP Trial / BAS Trial: https://account.hanatrial.ondemand.com

Next Steps

Reproduce each code block in a fresh CAP project in SAP Business Application Studio (cds init flight-demo), then paste your own screenshots and working code into the Session 4 'Your Code' section of the Demonstration Evidence Log — the same place your original flight-themed snippets already live.

When you're ready, use the Old vs New comparison table on page 2 as a drill: cover the right-hand column and try to write the CAP CDS syntax from memory, starting with @UI.LineItem and @UI.Facets, since those two appear in every Fiori Elements exam question.